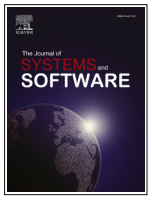
The Journal of Systems and Software 234 (2026) 112741



Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss





HiFlaky: Hierarchy-aware flakiness classification

Zheyuan Li [a], Zhenyu Wu [a], Yan Lei [a,b,*], Huan Xie [a], Maojin Li [a], Jian Hu [a]

[a] School of Big Data and Software Engineering, Chongqing University, Chongqing, 400044, China
[b] Peng Cheng Laboratory, Chongqing University, Shenzhen, 518066, China

ARTICLE INFO

Editor: Dr. Marcos Kalinowski

Keywords:
Flaky tests
Software testing
Hierarchy classification
Multi-label classification

ABSTRACT

Flaky tests present a major challenge in software testing, as they intermittently pass or fail without changes to the source code, leading to developer frustration and wasted resources. Current methods for detecting and classifying flaky tests often overlook the hierarchical dependencies among the root causes of flakiness in test code. Additionally, their tools fail to handle complex cases involving multiple flaky causes and lack the capability for fine-grained classification.

To this end, we propose HiFlaky, a hierarchy-aware multi-label classification method for flaky tests that identifies multiple co-occurring root causes along different hierarchical paths, such as diagnosing a single test with both "Flaky/NOD/Network" and "Flaky/NOD/Concurrency/Async Wait". HiFlaky leverages static semantic features from test code and hierarchy features, without dependency on dynamic execution features. To overcome the shortage of multi-root annotated data, we build a new dataset by expanding and labeling existing single-root datasets (IDoFT and FlakeFlagger) and 335 flaky tests from 145 Java projects on GitHub. Empirical evaluations demonstrate the effectiveness of HiFlaky in addressing both single and multi-root cause scenarios. In single-root cause scenarios, HiFlaky exhibits enhanced prediction accuracy compared to state-of-the-art methods, achieving a 30 % increase in Precision and an F1 score of 79 %. Furthermore, the fine-grained classification offered by HiFlaky provides useful insights that can facilitate root cause analysis, reduce debugging efforts, and contribute to improved software reliability. For complex multi-root cause scenarios, HiFlaky attains a Micro-F1 score of 0.812 and a Macro-F1 score of 0.414 across 29 categories.

1. Introduction

Software testing is a cornerstone of quality assurance, with its impact extending across the entire software lifecycle, from initial development to long-term maintenance and operations (Anand and Uddin, 2019). Throughout these phases, development teams rely on test outcomes as critical signals to guide decisions such as merging code, deploying new releases, or validating system stability (Pargaonkar, 2023). When a test reports a failure, developers typically assume that a defect exists in recent code changes and can begin debugging (Leung and White, 1989). However, in practice, developers often encounter a particular type of test, known as *flaky tests* (Luo et al., 2014), which may pass and fail non-deterministically across different executions, even when the code under test and the test itself, including its configuration. This inconsistency can negatively impact software development and reliability (Bishop and Cyra, 2010; Luo et al., 2014; Gruber et al., 2021; Hashemi et al., 2022; Tahir et al., 2023).

The identification of test flakiness poses a significant challenge in the automated software testing. Conducting an investigation into the nature and causes of flaky tests is a tedious and time-consuming task (Pirocanac, 2020; Eck et al., 2019; Kowalczyk et al., 2020; Labuschagne et al., 2017). For example, a 2016 report from Google (Micco, 2016) reveals that approximately 16 % of their tests exhibit some degree of flakiness. Eck et al. (Eck et al., 2019) further report that production builds of the Firefox web browser, which are affected by one or more flaky test failures, experience an average of 234 crash reports.

Motivated by the urgent need for detecting flakiness, a series of tools have been proposed and achieved promising results (Bell et al., 2018; Lam et al., 2019; Shi et al., 2016; Alshammari et al., 2021; King et al., 2018; Pinto et al., 2020; Camara et al., 2021a; Pontillo et al., 2021; Camara et al., 2021b; Haben et al., 2021; Fatima et al., 2024). For example, some early approaches (Bell et al., 2018; Lam et al., 2019; Shi et al., 2016) detect flakiness by rerunning the test suites under different configurations. However, as shown in a recent study (Alshammari et al., 2021), around two thirds scenarios need more than 1000 reruns, indicating that rerun-based detection is often associated with high overhead. To deal with this, recent researchers (King et al., 2018; Pinto et al., 2020; Camara et al., 2021a; Pontillo et al., 2021; Camara et al., 2021b;

* Corresponding author.
E-mail address: yanlei@cqu.edu.cn (Y. Lei).

https://doi.org/10.1016/j.jss.2025.112741
Received 15 March 2025; Received in revised form 28 July 2025; Accepted 5 December 2025
Available online 9 December 2025
0164-1212/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Haben et al., 2021; Akli et al., 2023; Fatima et al., 2022) have explored approaches to detect flaky tests based on static features instead of rerunning the test suites. However, these existing classification methods often fall short in providing developers with the fine-grained insights needed to effectively resolve flaky tests in complex scenarios. Considering this gap, subsequent researches (Luo et al., 2014; Parry et al., 2021; Tahir et al., 2023) move towards conducting root cause analysis of flaky tests. Based on these findings, recent approaches such as (Akli et al., 2023) employ fine-grained classifiers and adopt learning-based methods to automatically predict the underlying issues causing flaky failures.

While significant advancements have been made in flaky test root cause analysis, current methodologies often fall short of delivering the granular, actionable insights necessary for developers to effectively address flaky tests, particularly in complex scenarios. For example, simply categorizing a flaky test as Concurrency is insufficient for aiding developers in debugging and resolving the issue. A more fine-grained identification of specific causes, such as Async Wait, within the Concurrency category can help developers more precisely pinpoint the root cause. In particular, existing approaches often overlook two crucial characteristics of flaky tests:

1. Hierarchical Nature of Root Causes: Extant research (Luo et al., 2014; Parry et al., 2021; Gruber et al., 2021; Tahir et al., 2023) suggests that flaky test root causes are not uniformly structured but rather exhibit a hierarchical organization. This hierarchy ranges from broad categories such as Order Dependency (OD) and Non-Order Dependency (NOD) to more refined sub-categories. However, current flakiness classification methods often simplify the problem as a flat classification task, thereby neglecting this inherent hierarchical structure which could provide valuable diagnostic information.
2. Multi-Label Root Causes: Conventional approaches typically frame flakiness classification as a single-label problem. However, recent studies (Akli et al., 2023; Hashemi et al., 2022) indicate that flaky tests can frequently arise from the interplay of multiple interacting root causes. Consequently, single-label classification paradigms may prove inadequate in fully capturing the intricate nature of flaky test failures.

To illustrate these limitations, consider a scenario from the literature where a test reports a failure intermittently due to two co-occurring issues: a slow network response and an inadequate asynchronous wait (Hashemi et al., 2022). A basic detection tool might simply identify the test as “Flaky”. Even a more advanced single-label classifier like FlakyCat (Akli et al., 2023) would be forced to choose one category, perhaps “Network”. While informative, this singular label is incomplete and can lead to frustrating debugging cycles when the undiagnosed concurrency issue persists.

To deal with these challenges, we propose HiFlaky. For the same multi-cause scenario, it provides a complete, multi-label, hierarchical diagnosis identifying both root-cause paths: “Flaky/NOD/Network” and “Flaky/NOD/Concurrency/Async Wait”. Such granularity enables developers to directly focus their investigation on all contributing factors, substantially accelerating the debugging process.

To the best of our knowledge, there is no public dataset that includes multiple labels related to flaky root causes. To address this gap, the four of us independently annotate data from IDoFT (Lam, 2020), FlakeFlagger (Alshammari et al., 2021), and GitHub programs from Gruber et al. (Gruber et al., 2024). Any discrepancies in annotations were resolved through group discussions to ensure the reliability of the results, based on Cohen’s kappa coefficient (κ) (McHugh, 2012). By utilizing code mutation enhancement (Fatima et al., 2024), we ultimately get a multi-label dataset consisting of 10,790 instances related to flakiness.

Notably, our approach relies solely on static test code, rather than execution features, to perform effective fine-grained hierarchical classification of flaky tests. Previous research indicates that over 70 % of flaky test issues stem from the test code itself (Luo et al., 2014; Lam et al., 2020a; Zhang et al., 2023), and in large systems, white-box analysis of production source code may not be a feasible solution due to challenges in code accessibility and the cost of such deep analysis. (Fatima et al., 2022; Velez et al., 2021, 2020).

To evaluate the capability of HiFlaky to detect flakiness, we compared it with Flakify (Fatima et al., 2022) and FlakeFlagger (Alshammari et al., 2021) using the FlakeFlagger dataset. HiFlaky exhibits a 30 % precision increase and achieves a higher F1 score. Moreover, in multi-root cause scenarios, HiFlaky achieves a Micro-F1 score of 0.812 and a Macro-F1 score of 0.414. By identifying multiple root causes for each failing test, HiFlaky provides actionable insights that can substantially reduce debugging time and improve test maintainability in real-world development scenarios.

In summary, our work makes the following contributions:

- We investigate the integration of hierarchical features and semantic features to enhance the accuracy of flaky test detection.
- We extend the analysis of flaky tests from single-root cause scenarios to more complex multi-root cause scenarios.
- We empirically summarize common flaky causes and establish a hierarchical structure for them. Based on this, we build the first dataset containing multiple flaky causes, aggregated from three public datasets and encompassing 29 categories of flaky causes.
- We conduct experiments using widely-used benchmark, FlakeFlagger, to evaluate HiFlaky. The results demonstrate that HiFlaky achieves superior flaky test detection accuracy compared to two state-of-the-art baselines in terms of both Precision and F1 score, with a significant improvement (i.e., a 30 % increase in Precision and an F1 score boost to 79 %).

All implementations related to this study are publicly available,[1] with the expectation that our contributions will facilitate further advancements in both research and practical applications.

2. Background and motivation

2.1. Background

2.1.1. Detection of flakiness

In recent years, detecting flakiness has become a critical area of research in software testing, as flaky tests can significantly impact both developers and system reliability, making their detection essential (Luo et al., 2014; Micco, 2016; Gruber et al., 2021; Hashemi et al., 2022; Tahir et al., 2023). Flaky tests refer to cases that produce inconsistent results across different executions. Consequently, a common approach for detecting flakiness is to rerun the test suite. Several tools, including iDFlakes (Lam et al., 2019), NonDex (Gyori et al., 2016), DeFlaker (Bell et al., 2018), Shaker (Silva et al., 2020) and FlakeScanner, aim to detect flakiness by rerunning tests. However, rerunning the test suite is resource-intensive, and simply identifying whether a test is flaky does not help developers understand the root cause of the flakiness, which is essential for resolution.

2.1.2. Root cause of flakiness

With the growing emphasis on detecting flakiness, researchers have turned their attention to analyzing the root causes of flaky tests. Luo et al. (2014) conduct a comprehensive investigation of flaky root causes across 51 open-source projects and identified 10 common causes. Eck et al. (2019) then extend their work through the survey with developers, uncovering four additional causes. Since order-dependent flakiness can be detected automatically, flaky tests are often classified into order-dependent and non-order-dependent types. Parry et al. (2021) further investigate the causes of order-dependent tests. In recent years, research has delved deeper into the hierarchical classification of flaky root causes, with researchers providing a clear hierarchical classification of

[1] https://github.com/LichiY/HiFlaky

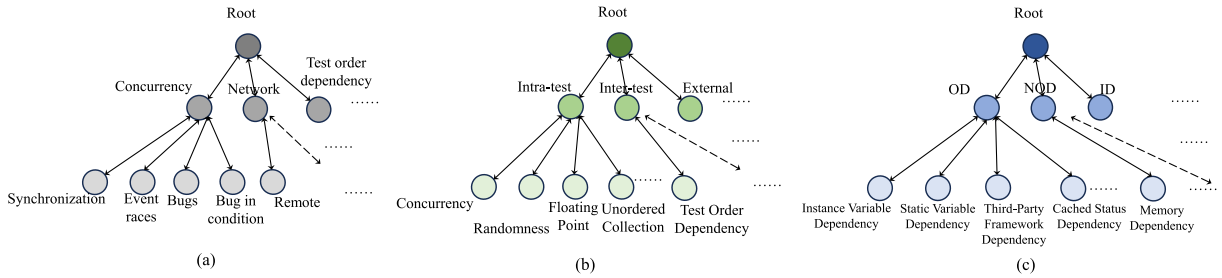


Fig. 1. Visualizations of hierarchical structures for flaky test root causes, based on the taxonomies described in (a) Tahir et al. (2023), (b) Parry et al. (2021), and (c) Chen et al. (2023).

```
server.on('stream', common.mustCall((stream) => {
    stream.on('error', common.mustCall(() => {
    stream.on('close', common.mustCall(() => {
```

Fig. 2. A test with multiple flaky causes taken from Node.js (Hashemi et al., 2022).

flaky root causes based on different criteria (Hashemi et al., 2022; Tahir et al., 2023). Unfortunately, this hierarchical information is typically overlooked in automated flaky test analysis. The hierarchical labels used in HiFlaky are derived from the categorization of recent studies (Chen et al., 2023; Tahir et al., 2023; Gruber et al., 2024).

2.1.3. Classification of flaky cases

As flaky root cause analysis has advanced, the focus has expanded beyond simply detecting flaky tests to classifying them based on their underlying causes. Several automated tools for classifying flaky tests have been developed in recent years (Akli et al., 2023; Fatima et al., 2024). FlakyCat (Akli et al., 2023) is one of the first approach to classify flaky tests based on their root cause categories, demonstrating effectiveness in this classification. FlakyFix (Fatima et al., 2024) presents an intriguing approach by defining new categories based on the specific parts of the test code that require fixes, rather than focusing on the flaky root cause itself. A precise, fine-grained classification of a flaky test's root cause provides immediate, actionable guidance for developers by significantly narrowing the problem space. This task is a critical and tractable prerequisite for the more challenging downstream goals of direct localization and automated repair. A robust classifier can serve as a foundational component for future tools that tackle these problems by enabling category-specific bug-finding and repair strategies.

2.2. Motivation

2.2.1. Hierarchy structure of flakiness.

With the advancement of more granular flaky root cause analysis, researchers have begun to gradually distinguish flaky root cause categories in a top-down manner. As shown in Fig. 1, researchs by Parry et al. (2021), Chen et al. (2023) and Tahir et al. (2023) both classify the flaky root causes based on the hierarchy structure.

However, recent work on automating the classification of flaky root causes often bypasses this hierarchical structure by directly predicting lower-level causes. As demonstrated in Gopal and Yang (2013a), hierarchies provide a natural framework for organizing and browsing unstructured data across multiple levels of granularity. Moreover, top-down root cause analysis not only enhances the accuracy of inferences by fully utilizing the dependencies between different root causes but also increases the interpretability of the reasoning process, thereby offering a more detailed diagnosis of flaky tests for both developers and researchers.

2.2.2. Multiple root causes of flakiness.

As revealed in FlakyCat (Akli et al., 2023), and (Hashemi et al., 2022), flakiness can arise from multiple reasons. For instance, as illustrated in an example[2] in Fig. 2 from Hashemi et al. (2022), both network-related issues and connection wait times contribute equally to test instability, resulting in tests being classified under both the network and asynchronous wait categories. Similar cases are also discussed in FlakyCat (Akli et al., 2023) However, existing automation tools treat the classification of flaky root causes as a single-label classification task, which limits their ability to handle complex scenarios where a test case is associated with multiple root causes simultaneously. A further consideration in such scenarios is whether a single dominant root cause exists that is more fundamental than the others. Prior work indicates that multiple, distinct root causes can be deeply entangled or contribute equally to a test's instability (Akli et al., 2023; Hashemi et al., 2022). In such situations, attempting to select a single dominant cause risks oversimplifying the problem and may obscure other critical factors from the developer.

To address these issues, we design a novel approach for classifying flaky root causes: HiFlaky , which aggregates hierarchical features and treats the problem as a multi-label classification task.

3. Approach

In this section, we will introduce our method HiFlaky , and demonstrate how to leverage both semantic features and structural characteristics of labels to perform hierarchical multi-label classification for flaky tests.

We first define the problem in Section 3.1, formalizing the multi-label hierarchical classification problem of flaky tests. Next, in Section 3.2, we provide an overview of the proposed model, describing its overall framework to facilitate a better understanding of the workflow. In Section 3.3, we discuss the data preprocessing steps, including the calculation of prior hierarchical information and the preprocessing of test source code. Then, in Section 3.4, we provide a detailed description of the feature extraction process, which includes the extraction of semantic features, hierarchy structural features, and the method for aggregating these features. Finally, in Section 3.5, we demonstrate how these features are used for classification, enabling the effective prediction of flaky tests.

3.1. Problem definition

In this section, we will outline the research objectives, thereby laying the theoretical foundation for the subsequent method development.

The label of flaky tests can be organized into a predefined hierarchical structure. These hierarchical relationships are based on the logical dependencies between labels, capturing the hierarchical dependencies among the labels.

[2] https://tinyurl.com/2p8hwj7w

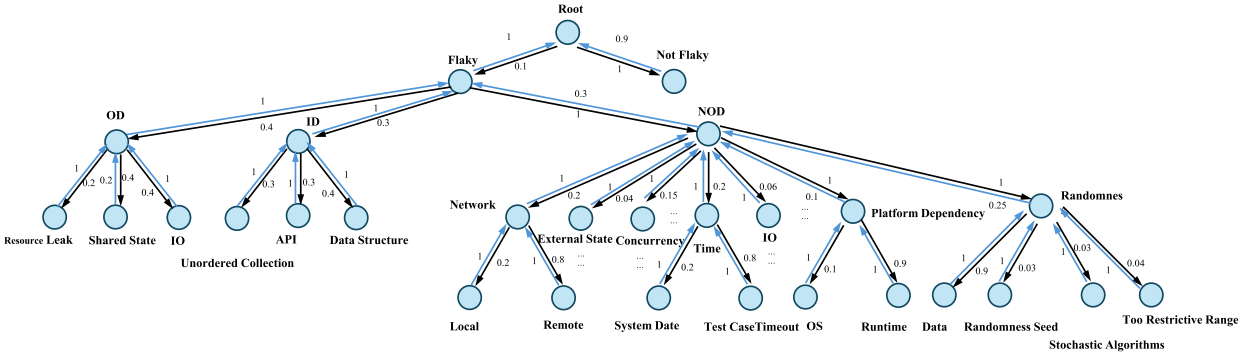


Fig. 3. Example of the taxonomic hierarchy. The number indicates the prior probability of label dependencies according to the training corpus.

We organize the labels into subsets based on the relationships between label categories, forming a tree-like or Directed Acyclic Graph (DAG) structure. It is important to note that the DAG structure can be simplified into a tree structure by distinguishing each label node as a unique path node. By this way, we consider the hierarchical label structure of flaky tests as a tree-like structure.

As illustrated in Fig. 3, we model the hierarchical label structure as a directed graph $G = (V, \overrightarrow{E}, \overleftarrow{E})$, where V represents the set of label nodes $V = \{v_1, v_2, \ldots, v_C\}$, and C denotes the total number of label nodes. The directed edges $\overrightarrow{E}$ capture the top-to-bottom hierarchical paths, while $\overleftarrow{E}$ capture the bottom-to-top paths. We formally define the hierarchical classification of flaky tests as $H = (X, L)$, where X represents a batch of test case code, and L denotes the corresponding ground truth labels. Each element $x_i \in X$ is a token sequence of a test case associated with a label set l_i, which contains multiple hierarchical labels relevant to x_i.

As depicted in Fig. 3, each label set l_i comprises categories that align with one or more sub-paths within the hierarchical structure. Importantly, if a sample is associated with a child node $v_j \in \text{child}(i)$, it is also implicitly associated with the parent node v_i. We empirically summarize common flaky causes and hierarchy structures from recent research (Gruber et al., 2021; Parry et al., 2021; Hashemi et al., 2022; Tahir et al., 2023; Gruber et al., 2024). The final tree-like structure developed for HiFlaky is shown in Fig. 3.

3.2. Overview

Fig. 4 provides an overview of HiFlaky. The model leverages a hierarchy-aware structure encoder to model the label dependencies, enabling the modeling of the hierarchical label structure from a global perspective. It utilizes fine-grained hierarchical information and aggregates static semantic features from test code. HiFlaky consists of five key components: data preprocessing, code semantic features extraction, hierarchical structure features extraction, information aggregation, and classification.

During the data preprocessing phase, we tokenize the code of test cases, and calculate the prior probabilities. Then, in the feature extraction phase, we extract both semantic code features and hierachy structural flaky features. Subsequently, mixed features aggregation is performed to integrate different types of features, creating a unified representation of features. In the classification phase, the aggregated features are used to predict flaky tests, validating the effectiveness of model.

Next, we will provide a detailed explanation of each part of HiFlaky.

3.3. Data pre-processing

3.3.1. Prior hierarchy information

The prior hierarchical information of flaky tests describes the hierarchical relationships between labels, and how to effectively leverage established structure is a key challenge. The method proposed in this paper utilizes prior knowledge of label dependencies, treating it as label correlation based on Bayesian statistical inference (Berger et al., 2009; Wang et al., 2021; Kass and Wasserman, 1996; Berger and Bernardo, 1992). The edge features between label nodes are represented as prior probabilities, which are used to aggregate information. Specifically, we use the prior probabilities of label dependencies as prior hierarchical knowledge. Suppose there exists a hierarchical path $e_{i,j}$ between a parent node v_i and a child node v_j, then the feature of this edge $f(e_{i,j})$ can represented by the following prior probability:

$$
\begin{aligned}
P(U_j \mid U_i) &= \frac{P(U_j \cap U_i)}{P(U_i)} = \frac{P(U_j)}{P(U_i)} = \frac{N_j}{N_i}, \\
P(U_i \mid U_j) &= \frac{P(U_i \cap U_j)}{P(U_j)} = \frac{P(U_j)}{P(U_j)} = 1.0,
\end{aligned}
\tag{1}
$$

Where U_k represents the occurrence of node v_k, $P(U_j \mid U_i)$, represents the conditional probability of node v_j occurring given that v_i occurs, and $P(U_j \cap U_i)$ is the joint probability of $\{v_j, v_i\}$ occurring together. Since in the hierarchical structure, the occurrence of a child node necessarily implies the occurrence of the parent node, N_k denotes the count of U_k in the training subset. We scale and normalize the prior probability of the child $v_{child(k)}$ node such that the total sum equals 1.

3.3.2. Flaky test tokenization

For effective semantic feature extraction, test code is initially tokenized. Tokenization, a fundamental preprocessing step for language models, transforms source code into a sequence of processable tokens. In this work, we employ a tokenizer based on Byte-Pair Encoding (BPE), a data compression algorithm widely adopted in natural language processing and demonstrably effective for code tokenization (Sennrich et al., 2015; Abram et al., 2012). BPE tokenization facilitates subword-level code representation, thereby addressing out-of-vocabulary concerns and enabling the capture of fine-grained code semantics. Subsequently, this tokenized representation serves as input for semantic feature extraction by the UniXcoder model (Guo et al., 2022).

3.4. Feature extraction

3.4.1. Semantic feature extraction

Recently, code embeddings generated by pre-trained language models have been widely adopted for source code representation (Fatima et al., 2022; Zhou et al., 2021). We selected the pre-trained UniXcoder model (Guo et al., 2022) to generate embeddings for the test code. UniXcoder has demonstrated excellent performance in understanding code semantics and is applicable to various tasks, such as code completion and code search.

As shown in Fig. 5, the overall UniXcoder model architecture can be divided into three parts: input embedding, UniXcoder learning, and semantic feature extraction. The token embedding and position embedding of the input sequence are represented by x_i and p_i, respectively. UniXcoder employs a BERT-based bidirectional transformer (Devlin,

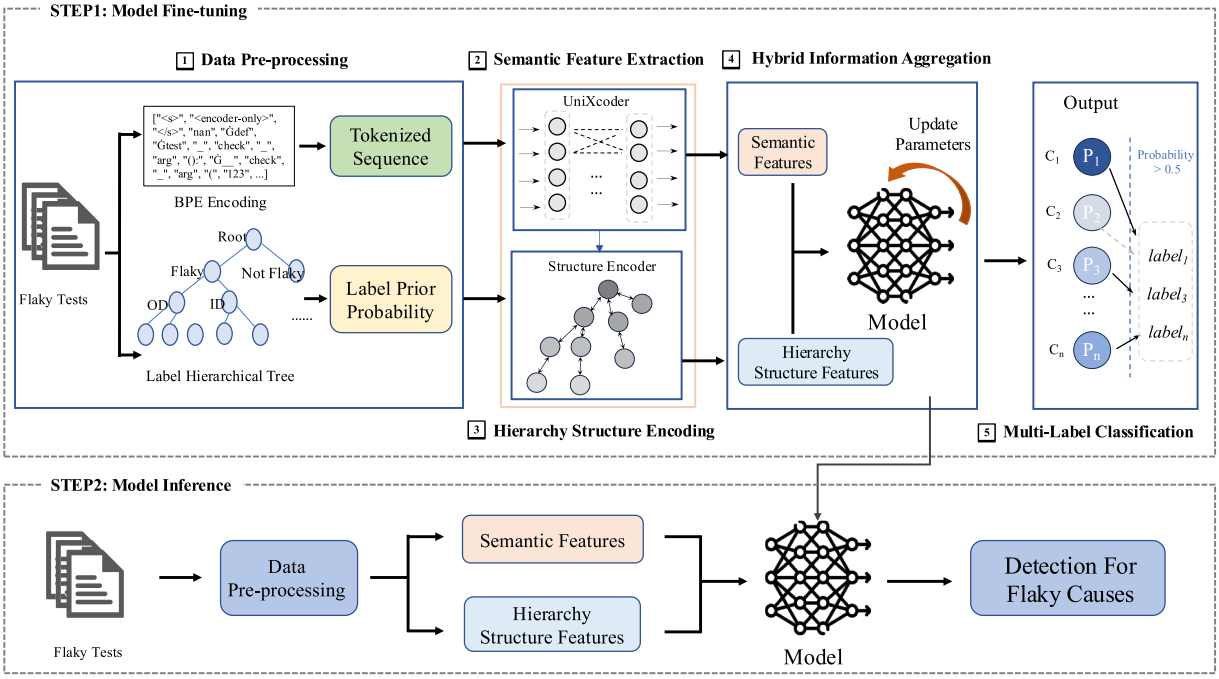


Fig. 4. The overview of HiFlaky.

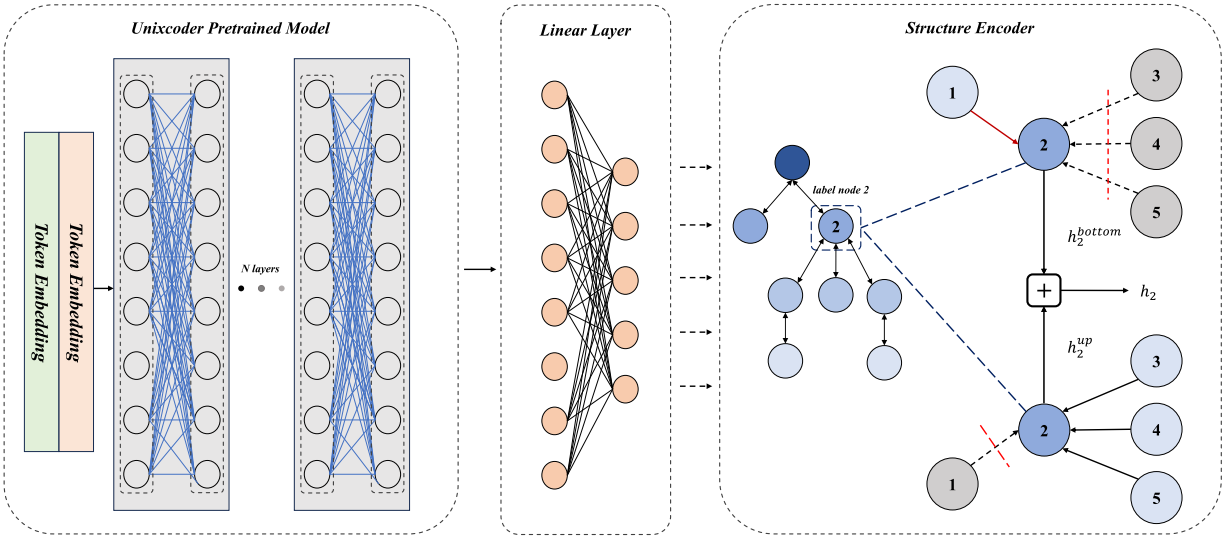


Fig. 5. The model architecture of semantic feature extraction.

2018) to provide contextual representations for each token in the input sequence. Each Transformer block contains a multi-head attention network and a fully connected feed-forward network to capture comprehensive global semantic information. Given the initial input $s_i^0 = [x_i, p_i]$, after N layers of Transformer blocks, the model output is the contextual representation of each token in the input sequence, denoted as s_i^N.

3.4.2. Hierarchy-aware structure encoder

As illustrated in Fig. 5, HiFlaky utilizes a hierarchy-aware structural encoder to model fine-grained hierarchical information. Specifically, in the top-down data flow, the prior probability $f_c(e_{i,j}) = \frac{N_j}{N_i}$ represents the strength of the relationship between nodes, while in the bottom-up data flow, $f_p(e_{i,j}) = 1.0$ ensures that information is propagated from lower-level labels to upper-level labels. Based on the prior hierarchical information computed during the preprocessing phase, HiFlaky employs a Bidirectional Tree-Long Short-Term Memory (Tree-LSTM) (Zhao and Hamarneh, 2019; Teng and Zhang, 2017) to combine the bottom-up information propagation from child nodes and the top-down information propagation from parent nodes. The node transformation is as follows:

$$\begin{aligned} \tilde{h}_k^{top} &= \sum_{j \in child(k)} f_p(e_{k,j}) h_j^{top}, \tilde{h}_k^{bottom} \quad = f_c(e_{k,p}) h_p^{bottom}, \\ h_k^{bi} &= h_k^{top} \oplus h_k^{bottom}, \end{aligned} \tag{2}$$

The hidden states h_k^{up} and h_k^{bottom} are calculated through the bottom-up and top-down approaches, respectively, denoted as $h_k = \text{TreeLSTM}(\tilde{h_k})$. The symbol $\oplus$ denotes the concatenation of hidden states, and the final hidden state of node k is represented by concatenating the bottom-up and top-down hidden states $h_k^{b_i}$.

3.4.3. Hybrid information aggregation

We developed an end-to-end hierarchy-aware global model to capture the interactions between the semantic features of test code and their

corresponding labels. By integrating the pre-trained UniXcoder model with a hierarchical structure encoder, HiFlaky generates label-aligned semantic representations of the code.

Given an input $x = (w_1, w_2, \ldots, w_n)$, UniXcoder is employed to extract semantic features, encoding the input sequence into a high-dimensional semantic embedding $S = (s_1, \ldots, s_n)$, where $s_i \in \mathbb{R}^{d_u}$ and d_u denotes the dimensionality of the semantic features.

In the context of multi-label hierarchical classification, managing complex relationships between code semantics and labels is essential. To facilitate this, HiFlaky reshapes the semantic features S into node inputs V using the following linear transformation:

$$V = SM, \tag{3}$$

where $M \in \mathbb{R}^{(n \times d_u) \times (C \times d_v)}$ is a trainable weight matrix, $S \in \mathbb{R}^{n \times d_u}$ represents the semantic feature matrix, and $V \in \mathbb{R}^{C \times d_v}$ are the node inputs after transformation. Here, d_v represents the dimensionality of each label node's representation vector. This transformation enables the model to leverage these semantic features for deeper propagation and updates within the hierarchy-aware encoder.

Within a predefined hierarchical structure, built from prior knowledge, each sample's semantic information is updated throughout the classification hierarchy. Mini-batch learning allows efficient updating of node representations within each batch. The initial node representation V is passed into the hierarchical encoder, and the resulting hidden state h embodies the hierarchy-aware semantic features, which are then used as input for the final classification layer.

This approach ensures that the hierarchy-aware structure encoder operates during both training and inference, facilitating the propagation of semantic information across hierarchical levels.

3.5. Classification

For the final classification, our model adopts a unified multi-label classification approach. The final hierarchy-aware features for every node are fed into a fully connected layer, which makes an independent prediction for each node. This approach ensures that an incorrect prediction for a general category does not cascade down and prevent correct predictions for its more specific descendants. The final layer operates on features that have already been enriched with hierarchical relationships, thereby preserving the structural information during classification. To further enforce these dependencies during parameter optimization, HiFlaky also incorporates a recursive regularization term (Gopal and Yang, 2013b) defined as:

$$L_r = \sum_{i \in C} \sum_{j \in child(i)} \frac{1}{2} \|w_i - w_j\|^2, \tag{4}$$

This regularization term is applied to the parameters of the final fully connected layer to maintain the dependency relationships among labels during parameter optimization. For multi-label classification, HiFlaky employs a binary cross-entropy loss function (Wehrmann et al., 2018; Nam et al., 2014; Wu et al., 2020):

$$L_c = -\sum_{i=1}^{N} \sum_{j=1}^{C} \left[y_{ij} \log(y'_{ij}) + (1 - y_{ij}) \log(1 - y'_{ij}) \right], \tag{5}$$

where y_{ij} and y'_{ij} represent the true label and the predicted sigmoid score for the jth label of the ith sample, respectively. The overall loss function is then defined as:

$$L_m = L_c + \lambda \cdot L_r \tag{6}$$

where λ is a weighting parameter that controls the influence of the regularization term.

4. Experiment setup

4.1. Dataset

Existing flaky test datasets, such as FlakeFlagger (Alshammari et al., 2021), typically contain single-label binary annotations (i.e., indicating only whether a test is flaky or not). These datasets lack the multi-label annotations necessary for more complex classification tasks. To bridge this gap, we developed a new multi-label dataset tailored to support HiFlaky in multi-label hierarchical classification tasks. This dataset was curated by selecting, expanding, and re-annotating data from existing sources, including the publicly available IDoFT dataset,[3] the FlakeFlagger dataset, and a selection of GitHub open-source projects identified by Gruber et. al.[4]

4.1.1. Datasets collection

Our dataset construction prioritized leveraging publicly accessible resources on flaky tests. The primary source of our flaky test examples was the International Dataset of Flaky Tests (IDoFT) (Lam, 2020), recognized as the largest and most frequently utilized dataset in the flaky test research domain. From IDoFT, we selected 768 test cases featuring developer-validated fixes for flakiness. This selection ensured a focus on real-world validated instances of flaky behavior. These test cases were drawn from 56 distinct projects, offering a diverse range of codebase contexts and flaky test scenarios. Secondarily, we incorporated flaky test data from GitHub projects as analyzed by Gruber et al. (2024). These projects were originally sourced from the Maven Central Repository and filtered to include only those hosted on GitHub and confirmed to use Maven as their build tool. The work by Gruber et al. offers a valuable collection of flaky tests identified and studied across diverse open-source GitHub projects. From their collection, we incorporated the 335 tests from 145 Java projects for which the original authors provided manual root-cause analysis and labels. These GitHub-sourced examples further enhanced our dataset's diversity, encompassing a broader spectrum of project characteristics and flaky test manifestations than IDoFT alone.

To ensure the combined dataset's integrity and uniqueness, a deduplication process was applied across data sources. Given the potential for overlapping projects and test cases between IDoFT and FlakeFlagger datasets, a deduplication step was implemented to remove redundant entries. This process primarily targeted instances where identical tests from the same project were present across different datasets.

To optimize the dataset for robust classification model training and facilitate effective data augmentation, non-flaky samples were also intentionally included. To establish a balanced dataset, we randomly selected and incorporated 5000 non-flaky test cases from the FlakeFlagger dataset, alongside a smaller set of 450 flaky test cases also sourced from FlakeFlagger.

4.1.2. Datasets labeling and augmentation

To maintain consistency and comprehensive labeling, we built a hierarchical tree structure of flaky test root causes based on prior research (Gruber et al., 2021; Parry et al., 2021; Hashemi et al., 2022; Tahir et al., 2023; Gruber et al., 2024). This tree was used as a framework for the hierarchical labeling process. High-level nodes were classified into NOD (Non-Order-Dependent), OD (Order-Dependent), and ID (Implementation-Dependent) categories, focusing on distinguishing test behaviors and trigger mechanisms. Although NOD and ID share a hierarchical relationship, ID tests were separated as a distinct category due to the unique challenges they pose and their higher prevalence in practice (Chen et al., 2023; Lam et al., 2020b).

Labeling was limited to the NOD, ID, and OD categories without including binary "Flaky" or "Not Flaky" labels. Four authors

[3] https://github.com/TestingResearchIllinois/idoft

[4] https://doi.org/10.6084/m9.figshare.22344706

Table 1
List of semantics-preserving mutation operators.

Operator	Description
Local Variable Renaming	Systematically rename local variable identifiers to a consistent pattern.
Equivalent Expression Replacement	Replace arithmetic or boolean expressions with logically equivalent but syntactically different forms.
Statement Reordering	Swap the order of two adjacent, independent statements (i.e., with no data dependency).
Control Flow Transformation	Convert control flow statements to a semantically equivalent structure.
Benign Code Insertion	Insert statements that have no side effects on the test's core logic, such as unused variable declarations.

independently annotated the dataset using project code, test code, failure information (e.g., stack traces and error logs), and pull request details to provide contextual insights. To ensure a unified understanding of root cause categories, we conducted an initial alignment exercise: 50 randomly selected samples were independently labeled, achieving a Cohen's kappa coefficient (κ) ranging from [0.79, 0.95], indicating high inter-rater agreement (McHugh, 2012). Discrepancies were resolved through group discussions until consensus was reached.

While our initial hierarchical tree was comprehensively constructed through analysis of prior research (Gruber et al., 2021; Parry et al., 2021; Hashemi et al., 2022; Tahir et al., 2023; Gruber et al., 2024), we encountered practical challenges during the labeling process. Several branches in the tree exhibited extremely sparse data distributions, with some containing either no instances or an insufficient number of samples (only 1–2 data points). These data-sparse branches presented significant challenges for effective model training and evaluation, as they could not be meaningfully distributed across training and test sets. Consequently, these sparsely populated branches were pruned from the hierarchical tree to ensure practical usability in experiments, while preserving their data at higher-level parent nodes. For example, instances originally labeled under fine-grained categories such as "NOD/Randomness/Too Restrictive Range" and "NOD/Randomness/Stochastic Algorithms" were retained under their parent category "NOD/Randomness". The final tree-like structure developed for HiFlaky consists of 29 distinct flakiness categories with a maximum depth of four levels. The complete taxonomy is available in our public repository.[5] This pruning is reflected in the distribution statistics presented in TABLE II, where certain theoretically possible categories are absent or consolidated with related categories to maintain sufficient data representation for meaningful evaluation.

After manual annotation, we applied a data augmentation technique inspired by mutation testing to create additional training samples. The goal of this process is to improve the model's robustness by learning from a wider variety of functionally equivalent test cases. To achieve this, we developed a script using the javalang library to parse the Java source code into an Abstract Syntax Tree. We then applied a set of semantics-preserving mutation operators, summarized in Table 1, to generate syntactic variations of the tests while keeping the original labels.

Before applying data augmentation, we split the dataset into training, validation, and test sets to ensure augmented samples were kept within the same subset as their originals, preventing overlap between training and testing data. This strategy mitigates data leakage and supports fair evaluation. Our data augmentation approach increased the training set size by fivefold, enhancing the model's robustness and generalizability for diverse and complex flaky test classification tasks. The statistics of our dataset are presented in Table 2. Table 3 presents a detailed breakdown of sample distribution statistics for the HiFlaky dataset across different flakiness root cause categories.

Table 2
Data Statistics: $|L|$ is the number of classes. Avg($|L_i|$) is the average number of classes per sample. Depth indicates the maximum level of hierarchy.

Dataset	$\|L\|$	Depth	Avg($\|L_i\|$)	Train	Val-	Test
Idoft	9	3	3.01	2300	750	790
FlakeFlagger	2	1	1.00	3270	1090	1090
ATG	29	4	3.39	1385	135	155
Total	29	4	2.10	6955	1975	2035

Table 3
Sample distribution statistics of HiFlaky dataset categories after data augmentation.

Category		Train	Val-	Test
Flaky	–	3155	725	755
Flaky/ID	–	915	140	145
	API Behavior Differences	760	100	115
	Data Structure Implementation Differences	125	30	20
	Unordered Collection Access	30	10	10
Flaky/NOD	–	895	225	260
	Concurrency	170	45	50
	Concurrency/Async Wait	155	40	45
	Concurrency/Dead Lock	15	5	5
	IO	65	20	20
	Network	155	40	50
	Network/Local	30	10	15
	Network/Remote	125	30	35
	Other	55	10	10
	Other/Program Logic	25	5	5
	Other/Floating Point Operations	20	5	5
	Platform Dependency	95	15	20
	Platform Dependency/Runtime	75	10	15
	Platform Dependency/OS	20	5	5
	Randomness	150	40	45
	Randomness/Data	140	40	45
	Time	205	55	65
	Time/System Date	45	10	15
	Time/Test Case Timeout	160	45	50
Flaky/OD	–	1075	270	260
	IO	265	75	55
	Resource Leak	255	75	85
	Shared State	555	120	120
Not Flaky	–	3800	1250	1280
Total		6955	1975	2035

4.2. Experimental settings

In this study, HiFlaky was implemented using the PyTorch framework and executed on an Ubuntu 20.04 server configured with a 40-core 2.4GHz Intel Xeon CPU, 252GB of RAM, and dual NVIDIA GeForce RTX 3090 GPUs (each with 24GB of VRAM).

We employ UniXcoder-base as the pre-trained model for extracting 768-dimensional semantic embeddings to capture the contextual information of the test code. For hierarchical label modeling, a bidirectional Tree-LSTM with 768-dimensional hidden units was used to learn dependencies between labels.

To mitigate overfitting, a dropout rate of 0.5 was applied at the embedding layer and 0.05 during node transformation. The training process utilized the Adam optimizer with a batch size of 64, a learning rate of 0.0001, and an early stopping mechanism. The learning rate decay factor was set to 1.0 with a patience of 5 epochs. Binary Cross-Entropy

[5] https://github.com/LichiY/HiFlaky

with Logits Loss (BCEWithLogitsLoss) was employed as the classification loss function, alongside recursive regularization with a penalty coefficient of 1×10^{-6}. Model evaluation was performed on the test subset using the model checkpoint that achieved the highest Micro-F1 score on the validation set during training.

4.3. Evaluation metrics

Following prior research (Akli et al., 2023; Alshammari et al., 2021; Fatima et al., 2022), we adopted widely-used metrics to evaluate the effectiveness of existing flaky test detection methods for single root cause scenarios, as detailed below:

- **Precision** measures the proportion of true positive flaky tests among all tests predicted as flaky, assessing the model's accuracy in single root cause detection. It is formally defined as shown in Eq. (7):

$$Precision = \frac{|T_{flaky} \cap \hat{T}_{flaky}|}{|\hat{T}_{flaky}|} \tag{7}$$

  where T_{flaky} represents the set of actual flaky tests, and $\hat{T}_{flaky}$ represents the set of tests predicted as flaky by the model.
- **Recall** evaluates the proportion of actual flaky tests that are correctly identified by the model, reflecting its sensitivity. It is formally defined in Eq. (8):

$$Recall = \frac{|T_{flaky} \cap \hat{T}_{flaky}|}{|T_{flaky}|} \tag{8}$$

- **F1-score** represents the harmonic mean of Precision and Recall and is used to provide a balanced evaluation of the model's detection performance in single root cause scenarios. It is formally defined by Eq. (9):

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{9}$$

Additionally, to evaluate the performance of HiFlaky in multi-label hierarchical classification tasks, we use Micro-F1 and Macro-F1 as the primary evaluation metrics. These metrics are commonly applied in multi-label and hierarchical classification tasks (Gopal and Yang, 2013a; Tang et al., 2009; Wu et al., 2016; Huang et al., 2016) and effectively assess the model's performance across different label categories:

- **Micro-F1** aggregates predictions over all labels and test instances by treating each (test, label) pair as a binary decision. Let T^{all}_{flaky} denote the set of all true (test, label) pairs across the root-cause hierarchy, and let $\hat{T}^{\text{all}}_{flaky}$ denote the set of all predicted (test, label) pairs. Micro-F1 is then defined as shown in Eq. (10):

$$\text{Micro-F1} = \frac{2 \times \left|T^{\text{all}}_{flaky} \cap \hat{T}^{\text{all}}_{flaky}\right|}{\left|T^{\text{all}}_{flaky}\right| + \left|\hat{T}^{\text{all}}_{flaky}\right|} \tag{10}$$

- **Macro-F1** calculates the F1 score for each label and then averages them, reflecting the model's balanced performance across all categories. It is particularly useful for assessing the model's effectiveness on infrequent labels. Macro-F1 is defined in Eq. (11):

$$\text{Macro-F1} = \frac{1}{|L|} \sum_{i=1}^{|L|} F_{1i} \tag{11}$$

  where $|L|$ represents the total number of labels, and F_{1i} denotes the F1 score for each label.

4.4. Baselines

To the best of our knowledge, we are the first to apply multi-label hierarchical classification to flaky tests. However, in order to better evaluate the performance of HiFlaky , we compare HiFlaky with several current state-of-the-art flaky detection methods in single-root cause scenarios.

FlakeFlagger Alshammari et al. (2021) proposed a machine learning-based prediction method that combines both static and dynamic features to predict flaky test. Specifically, FlakeFlagger utilizes features such as test execution time, coverage, third-party library usage, and test smells. It applies Information Gain (IG) to filter features, selecting only those with an IG value greater than 0.01 for training. In this experiment, we reproduced the results of Alshammari et al. using the reproduction package they provided. FlakeFlagger is one of the best state-of-the-art ML-based predictor for flaky test cases (Fatima et al., 2022).

Flakify Fatima et al. (2022) introduced a prediction method based on a black-box language model, leveraging CodeBERT (a pre-trained transformer model) to generate embeddings for test code and predict test instability. Unlike FlakeFlagger, Flakify does not require access to production code; it relies entirely on the syntax and semantic information of the test code. We chose Flakify as a baseline because it demonstrated high precision and recall across multiple datasets, showing strong cross-project generalization capabilities.

4.5. Results

To comprehensively evaluate the effectiveness of HiFlaky , we investigate the following research questions:

- **RQ1: How does HiFlaky perform in detecting flaky tests compared to existing state-of-the-art methods?**
  This research question aims to assess the performance of HiFlaky in detecting flaky tests with a single root cause and to compare it with the leading baseline methods. We report precision, recall, and F1-scores on the FlakeFlagger dataset to validate the accuracy and robustness of HiFlaky in this context.
- **RQ2: How does HiFlaky perform in multi-label hierarchical scenarios?**
  This research question examines HiFlaky 's capability in multi-label classification for scenarios with multiple root causes. We report the performance of HiFlaky using multi-label evaluation metrics such as Micro-F1 and Macro-F1 to demonstrate its hierarchical modeling ability in classifying flaky tests with diverse root causes.
- **RQ3: What is the contribution of each component of HiFlaky to overall performance?**
  This question investigates the contribution of various components within HiFlaky through ablation studies. We analyze the impact of different feature extraction models (e.g., UniXcoder versus CodeBERT) and the hierarchical modeling module (with and without the hierarchical encoder). By presenting results from different combinations of features and model components, we validate the effectiveness of design in combining semantic features and hierarchical features.

5. Result analysis

5.1. RQ1 results

To evaluate the effectiveness of HiFlaky in detecting flaky tests, we compared its performance against state-of-the-art baseline models. We hypothesize that representations effective in binary classification (Flaky vs. Not Flaky) could also be beneficial for categorizing tests according to their root causes. In this section, we reproduce the result of Flakify and FlakeFlagger on the FlakeFlagger dataset and compare its performance with HiFlaky . Table 4 summarizes predictive performance of HiFlaky on the FlakeFlagger dataset and compares it with the baseline models, FlakeFlagger and Flakify, across metrics such as precision, recall, and F1-score.

HiFlaky is inherently a multi-label hierarchical classification model, not originally designed for binary classification. However, to facilitate

Table 4
Comparison of predictive performance of HiFlaky with baseline models (FlakeFlagger and Flakify) on the FlakeFlagger dataset, measured using precision, recall, and F1-score.

Approach	Precision	Recall	F1
HiFlaky	**85 %**	77 %	**79 %**
Flakify	65 %	**85 %**	74 %
FlakeFlagger	61 %	73 %	66 %

a comparison with existing flaky test detection models, we adapted HiFlaky to binary classification by mapping the highest prediction probability to its respective label, determining whether a test is flaky or not. This mapping allows the model to classify tests based on the highest predicted probability.

The results show that HiFlaky outperforms baseline models, achieving a significant improvement in both precision and F1-score for detecting flaky and non-flaky tests. Notably, HiFlaky 's precision surpassed the best-performing baseline by 30 %, demonstrating its superior detection capabilities.

These findings indicate that the hierarchical modeling approach is not only beneficial for fine-grained root cause analysis also essential for capturing deeper representations of flaky test code, even in basic binary classification tasks. The hierarchical structure of HiFlaky likely enables the system to recognize structural features of flaky tests, which are often overlooked by models employing flatter representations. Therefore, incorporating hierarchical information, can significantly enhance the performance of flakiness detection models, regardless of the granularity of classification.

Despite achieving high precision and F1-score, HiFlaky demonstrated a comparatively lower recall for the Flaky category. This indicates that HiFlaky tends towards a more conservative prediction approach, prioritizing the accuracy of positive flaky classifications over maximizing the detection of all flaky tests. Although this conservative prediction strategy leads to a slightly lower recall rate, developers in practical flaky test detection typically prioritize minimizing false positives to avoid wasting debugging efforts (Winter et al., 2023; Memon et al., 2017; Tao et al., 2014).

In conclusion, while HiFlaky was not initially conceived for binary flakiness detection, its application to this task demonstrates promising performance. Our findings reveal that HiFlaky achieves superior precision and F1-score compared to state-of-the-art baselines, indicating the effectiveness of its hierarchical architecture.

5.2. RQ2 results

To assess the performance of HiFlaky in a multi-label hierarchical classification setting, we evaluate it on the test dataset described in Section 4.1, which includes tests with diverse root causes, and analyze the performance metrics for each category. The results are shown in Table 5.

HiFlaky exhibits strong overall performance in multi-label classification, as indicated by a Micro-F1 score of 0.812. This notable Micro-F1 value suggests that HiFlaky attains good general accuracy across all test instances, successfully classifying flaky tests into their respective hierarchical categories. The Micro-F1 metric, being less susceptible to class imbalance, further indicates that HiFlaky demonstrates a relatively consistent ability to classify flaky tests, especially within the prevalent categories in our dataset.

The Macro-F1 score of 0.414, while lower than the Micro-F1 score, highlights a crucial challenge in hierarchical flaky test classification: the difficulty posed by infrequent categories and the long-tailed distribution of flaky root causes. Given that Macro-F1 is more sensitive to performance on less frequent categories, its reduced value underscores potential limitations of HiFlaky in accurately classifying these less common, yet often highly specific, root causes. This observation is further supported by the category-level performance presented in Table 5, where several categories with limited training data show significantly lower performance. This performance variation does not necessarily represent a deficiency of HiFlaky , but rather reflects the challenge of data scarcity in the long-tail distribution of flaky root causes. Machine learning models, including HiFlaky , typically encounter difficulties in generalizing to categories with limited training instances.

Table 5
Classification performance of HiFlaky across multiple flaky root causes.

Category		Precision	Recall	F1
Flaky	–	**96 %**	**93 %**	**95 %**
Flaky/ID	–	71 %	**77 %**	**74 %**
	API Behavior Differences	**92 %**	62 %	74 %
	Data Structure Implementation Differences	23 %	35 %	28 %
	Unordered Collection Access	14 %	32 %	19 %
Flaky/NOD	–	89 %	70 %	78 %
	Concurrency	50 %	40 %	44 %
	Concurrency/Async Wait	53 %	40 %	46 %
	Concurrency/Dead Lock	46 %	32 %	38 %
	IO	75 %	15 %	25 %
	Network	70 %	56 %	62 %
	Network/Local	**100 %**	**80 %**	**89 %**
	Network/Remote	**100 %**	29 %	44 %
	Other	77 %	33 %	47 %
	Other/Program Logic	50 %	20 %	29 %
	Other/Floating Point Operations	42 %	31 %	36 %
	Platform Dependency	80 %	27 %	40 %
	Platform Dependency/Runtime	100 %	27 %	42 %
	Platform Dependency/OS	52 %	27 %	36 %
	Randomness	59 %	53 %	56 %
	Randomness/Data	64 %	45 %	53 %
	Time	46 %	40 %	43 %
	Time/System Date	**100 %**	33 %	50 %
	Time/Test Case Timeout	34 %	24 %	28 %
Flaky/OD	–	80 %	80 %	80 %
	IO	58 %	27 %	37 %
	Resource Leak	**81 %**	40 %	54 %
	Shared State	78 %	**88 %**	**83 %**
Not Flaky	–	**94 %**	**98 %**	**96 %**

HiFlaky demonstrates particularly effective performance, exhibiting consistently strong Precision, Recall, and F1-scores, in broader, higher-level categories such as "Flaky" (F1: 95 %), "Flaky/NOD" (F1: 78 %), and "Flaky/OD" (F1: 80 %), as well as in relevant mid-level categories including "Network/Local" (F1: 89 %) and "Shared State" (F1: 83 %). These results validate the effectiveness of our hierarchical approach, providing valuable granularity that directly benefits developers during debugging efforts.

The performance across different hierarchical levels reveals an important characteristic of fine-grained classification systems. As expected in hierarchical classification, we observe a natural trade-off between classification granularity and performance metrics in the most specific categories. For instance, specialized categories with inherently smaller representation in the dataset such as "Data Structure Implementation Differences" (F1: 28 %), "Unordered Collection Access" (F1: 19 %), and "IO" (F1: 25 %) present greater classification challenges. This pattern is consistent with established machine learning principles and does not diminish the value of HiFlaky 's fine-grained approach. Rather, it highlights the complementary nature of HiFlaky 's multi-level classifications, where developers can leverage both highly reliable broader classifications and more detailed fine-grained insights when sufficient evidence exists. This balanced hierarchical approach offers developers significant flexibility - providing both highly confident guidance at higher levels while still offering valuable fine-grained insights where possible,

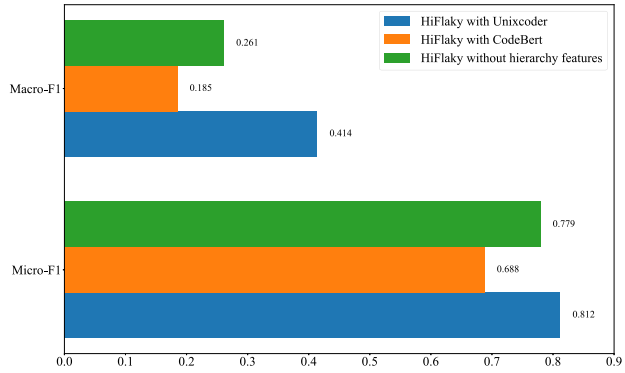


Fig. 6. Ablation study of HiFlaky with different configurations.

ultimately enhancing debugging efficiency across diverse flaky test scenarios.

Overall, the performance of HiFlaky in multi-label hierarchical classification is robust, particularly in achieving high Micro-F1 scores. This performance demonstrates its ability to leverage label dependencies to improve detection, especially in complex multi-label and multi-root cause scenarios. The results also indicate that addressing data scarcity and the long-tail distribution of flaky root causes will be an important future work.

5.3. RQ3 results

We conduct an ablation study to quantify the contribution of HiFlaky's individual components. This involves systematically removing key architectural parts, such as the hierarchy-aware encoder, to measure their impact on the model's final performance and thereby validate our design choices. Specifically, we substitute UniXcoder with CodeBERT as the text feature extractor and test a variant of the model without the hierarchical structure encoder. These experiments help clarify the contribution of each component. The results are depicted in Fig. 6.

HiFlaky with UniXcoder. The complete version of HiFlaky, using UniXcoder and the hierarchical structure encoder, achieve the highest performance, with Micro-F1 and Macro-F1 scores of 0.812 and 0.414, respectively. This demonstrates high overall classification accuracy (Micro-F1) and balanced performance (Macro-F1) across categories, although it shows slightly weaker performance on some rare categories. The hierarchical structure enabled the model to capture dependencies between high-level categories and subcategories, significantly enhancing the detection of common categories and overall classification efficacy.

HiFlaky without hierarchy features.

In the absence of the hierarchical structure encoder, Micro-F1 dropped from 0.812 to 0.780 and Macro-F1 from 0.414 to 0.261. This highlights the critical role of the hierarchical structure in modeling relationships between labels. Without this component, the model lost its capability to understand label dependencies, resulting in a marked decline in performance, particularly for infrequent categories, as indicated by the substantial drop in Macro-F1. It confirms our core hypothesis: modeling hierarchical relationships among flaky root causes can facilitate inference of fine-grained flaky root causes. Without the hierarchical encoder, HiFlaky loses its ability to leverage label dependencies and effectively differentiate between categories, particularly the rarer, more specific ones, as evidenced by the drastic Macro-F1 decline.

HiFlaky with CodeBERT. Replacing UniXcoder with CodeBERT resulted in a Micro-F1 of 0.688 and a Macro-F1 of 0.185. CodeBERT was less effective at capturing semantic information in test code compared to UniXcoder, especially in multi-label tasks. The superior understanding of UniXcoder for contextual code semantics enable HiFlaky to learn and differentiate between hierarchical labels more accurately. This enhanced semantic representation contributed to improved overall classification performance.

In summary, these ablation studies demonstrate the crucial role of HiFlaky's hierarchical structure and the advanced text feature extraction capabilities of UniXcoder. The hierarchical structure enable the model to learn shared features across related labels, improving accuracy and recall in complex multi-label tasks. The advanced semantic understanding of UniXcoder further enhanced the model's ability to represent features, contributing to its strong performance in multi-label classification.

6. Threats to validity

6.1. Construct validity

Construct validity concerns whether our analysis accurately measures the intended research objectives. This study aims to support multi-label hierarchical classification tasks by constructing a new multi-label dataset. However, the complexity and subjectivity of the data labeling process may affect label accuracy. Although independent labeling by multiple researchers and the verification of consistency using Cohen's kappa coefficient (with κ values ranging from [0.79, 0.95], indicating strong agreement) were employed, there may still be instances of label bias or errors. Additionally, the construction of the hierarchical label structure was based on existing literature and expert judgment, which could lead to incomplete or overlooked categorizations. Future studies should further refine labeling standards and methods to mitigate these threats.

6.2. Internal validity

Internal validity threats relate to the reliability of experimental results. While HiFlaky demonstrated strong performance in multi-label tasks, the limited sample size for certain categories in the dataset may have impacted model performance. To enhance the generalization of the model for low-sample categories, we used mutation testing techniques to augment the dataset. However, this approach may not fully capture the behavior of flaky tests in real-world scenarios.

6.3. External validity

External validity threats pertain to the generalizability of the study's findings. The dataset used in this study primarily consisted of Java projects, which, although varied in size and complexity, may limit the applicability of HiFlaky to other programming languages. While UniXcoder has demonstrated advantages in understanding multi-language semantics, the root cause distribution and characteristics of flaky tests can vary across different programming languages. To enhance generalizability, we collected data from multiple sources; however, future research should extend this validation to other programming languages and diverse projects. Additionally, the multi-label annotation strategy and the hierarchical structure encoder design employed in this study may perform differently in other contexts and require further validation across varied scenarios.

7. Conclusion

In this work, we present HiFlaky, a robust framework for multi-label hierarchical classification of flaky tests. By integrating semantic and structural label information, HiFlaky effectively captures label dependencies and enriches feature representations for complex testing scenarios. Our evaluation demonstrates that HiFlaky surpasses state-of-the-art models, including FlakeFlagger and Flakify, in precision and F1-score for detecting flaky tests. The hierarchical structure of HiFlaky effectively captures inter-label relationships, improving classification performance across diverse root causes and subcategories. The notable Micro-F1 score

of HiFlaky demonstrates its capability in identifying common flaky test categories, while its Macro-F1 scores reflect robust adaptability in handling infrequent categories, despite challenges in rare subcategories. We further conduct the ablation study to validate the architectural choices behind HiFlaky and demonstrate its robustness in multi-label flaky test classification.

Future work will involve expanding and diversifying the dataset to improve HiFlaky's generalizability and robustness in classifying flaky tests. Additionally, using HiFlaky's classification insights for automated flaky test repair is a promising direction, where root cause identification can aid frameworks in suggesting or implementing automated fixes, enhancing software reliability and streamlining testing.

CRediT authorship contribution statement

Zheyuan Li: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Zhenyu Wu:** Writing – review & editing, Writing – original draft, Investigation; **Yan Lei:** Writing – review & editing, Supervision, Resources, Funding acquisition; **Huan Xie:** Writing – review & editing; **Maojin Li:** Writing – review & editing; **Jian Hu:** Writing – review & editing.

Data availability

I have shared the link to my data and code at the manuscript.

Declaration of competing interest

None.

Acknowledgements

This research is supported by the Chongqing Technology Innovation and Application Development Special Project (CSTB2023TIAD-STX0029) and the National Natural Science Foundation of China (No. 62272072).

References

Abram, H., Earl, T.B., Su, Z., Gabel, M., Devanbu, P., 2012. On the naturalness of software. In: Proceedings of the 34th International Conference on Software Engineering. IEEE Computer Society, pp. 837–847.

Akli, A., Haben, G., Habchi, S., Papadakis, M., Le Traon, Y., 2023. FlakyCat: predicting flaky tests categories using few-shot learning. In: 2023 IEEE/ACM International Conference on Automation of Software Test (AST). IEEE, pp. 140–151.

Alshammari, A., Morris, C., Hilton, M., Bell, J., 2021. FlakeFlagger: predicting flakiness without rerunning tests. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). IEEE, pp. 1572–1584.

Anand, A., Uddin, A., 2019. Importance of software testing in the process of software development. Int. J. Sci. Res. Dev. 6 (12), 141–145.

Bell, J., Legunsen, O., Hilton, M., Eloussi, L., Yung, T., Marinov, D., 2018. DeFlaker: automatically detecting flaky tests. In: Proceedings of the 40th International Conference on Software Engineering, pp. 433–444.

Berger, J.O., Bernardo, J.M., 1992. On the development of the reference prior method. Bayesian Stat. 4 (4), 35–60.

Berger, J.O., Bernardo, J.M., Sun, D., 2009. The formal definition of reference priors. Ann. Statist. 37 (2), 905–938.

Bishop, P., Cyra, L., 2010. Overcoming non-determinism in testing smart devices: a case study. In: Computer Safety, Reliability, and Security: 29th International Conference, SAFECOMP 2010, Vienna, Austria, September 14–17, 2010. Proceedings 29. Springer, pp. 237–250.

Camara, B., Silva, M., Endo, A., Vergilio, S., 2021a. On the use of test smells for prediction of flaky tests. In: Proceedings of the 6th Brazilian Symposium on Systematic and Automated Software Testing, pp. 46–54.

Camara, B. H.P., Silva, M. A.G., Endo, A.T., Vergilio, S.R., 2021b. What is the vocabulary of flaky tests? An extended replication. In: 2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC). IEEE, pp. 444–454.

Chen, Y., Yildiz, A., Marinov, D., Jabbarvand, R., 2023. Transforming test suites into croissants. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1080–1092.

Devlin, J., 2018. BERT: pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

Eck, M., Palomba, F., Castelluccio, M., Bacchelli, A., 2019. Understanding flaky tests: the developer's perspective. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 830–840.

Fatima, S., Ghaleb, T.A., Briand, L., 2022. Flakify: a black-box, language model-based predictor for flaky tests. IEEE Trans. Software Eng. 49 (4), 1912–1927.

Fatima, S., Hemmati, H., Briand, L., 2024. FlakyFix: using large language models for predicting flaky test fix categories and test code repair. IEEE Trans. Software Eng. 50 (12), 3146–3171.

Gopal, S., Yang, Y., 2013a. Recursive regularization for large-scale classification with hierarchical and graphical dependencies. In: Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. Association for Computing Machinery, New York, NY, USA, p. 257–265. https://doi.org/10.1145/2487575.2487644.

Gopal, S., Yang, Y., 2013b. Recursive regularization for large-scale classification with hierarchical and graphical dependencies. In: Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 257–265.

Gruber, M., Lukasczyk, S., Kroiß, F., Fraser, G., 2021. An empirical study of flaky tests in python. In: 2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST). IEEE, pp. 148–158.

Gruber, M., Roslan, M.F., Parry, O., Scharnböck, F., McMinn, P., Fraser, G., 2024. Do automatic test generation tools generate flaky tests? In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–12.

Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J., 2022. UnixCoder: unified cross-modal pre-training for code representation. arXiv preprint arXiv:2203.03850.

Gyori, A., Lambeth, B., Shi, A., Legunsen, O., Marinov, D., 2016. NonDex: a tool for detecting and debugging wrong assumptions on java API specifications. In: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 993–997.

Haben, G., Habchi, S., Papadakis, M., Cordy, M., Le Traon, Y., 2021. A replication study on the usability of code vocabulary in predicting flaky tests. In: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR). IEEE, pp. 219–229.

Hashemi, N., Tahir, A., Rasheed, S., 2022. An empirical study of flaky tests in javascript. In: 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, pp. 24–34.

Huang, J., Li, G., Huang, Q., Wu, X., 2016. Learning label-specific features and class-dependent labels for multi-label classification. IEEE Trans. Knowl. Data Eng. 28 (12), 3309–3323.

Kass, R.E., Wasserman, L., 1996. The selection of prior distributions by formal rules. J. Am. Stat. Assoc. 91 (435), 1343–1370.

King, T.M., Santiago, D., Phillips, J., Clarke, P.J., 2018. Towards a Bayesian network model for predicting flaky automated tests. In: 2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C). IEEE, pp. 100–107.

Kowalczyk, E., Nair, K., Gao, Z., Silberstein, L., Long, T., Memon, A., 2020. Modeling and ranking flaky tests at apple. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice, pp. 110–119.

Labuschagne, A., Inozemtseva, L., Holmes, R., 2017. Measuring the cost of regression testing in practice: a study of java projects using continuous integration. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, pp. 821–830.

Lam, W., 2020. International dataset of flaky tests (IDoFT). http://mir.cs.illinois.edu/flakytests.

Lam, W., Muşlu, K., Sajnani, H., Thummalapenta, S., 2020a. A study on the lifecycle of flaky tests. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 1471–1482.

Lam, W., Oei, R., Shi, A., Marinov, D., Xie, T., 2019. iDFlakies: a framework for detecting and partially classifying flaky tests. In: 2019 12th IEEE Conference on Software Testing, Validation and Verification (iCST). IEEE, pp. 312–322.

Lam, W., Winter, S., Wei, A., Xie, T., Marinov, D., Bell, J., 2020b. A large-scale longitudinal study of flaky tests. Proc. ACM Prog. Lang. 4 (OOPSLA), 1–29.

Leung, H. K.N., White, L., 1989. Insights into regression testing (software testing). In: Proceedings. Conference on Software Maintenance-1989. IEEE, pp. 60–69.

Luo, Q., Hariri, F., Eloussi, L., Marinov, D., 2014. An empirical analysis of flaky tests. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 643–653.

McHugh, M.L., 2012. Interrater reliability: the kappa statistic. Biochem. Med. 22 (3), 276–282.

Memon, A., Gao, Z., Nguyen, B., Dhanda, S., Nickell, E., Siemborski, R., Micco, J., 2017. Taming google-scale continuous testing. In: 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP). IEEE, pp. 233–242.

Micco, J., 2016. Flaky tests at google and how we mitigate them. Online] https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html .

Nam, J., Kim, J., Loza Mencía, E., Gurevych, I., Fürnkranz, J., 2014. Large-scale multi-label text classification-revisiting neural networks. In: Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2014, Nancy, France, September 15–19, 2014. Proceedings, Part II 14. Springer, pp. 437–452.

Pargaonkar, S., 2023. A study on the benefits and limitations of software testing principles and techniques: software quality engineering. Int. J. Sci. Res. Publ. 13 (8), 149–155.

Parry, O., Kapfhammer, G.M., Hilton, M., McMinn, P., 2021. A survey of flaky tests. ACM Trans. Software Eng. Methodol. (TOSEM) 31 (1), 1–74.

Pinto, G., Miranda, B., Dissanayake, S., d'Amorim, M., Treude, C., Bertolino, A., 2020. What is the vocabulary of flaky tests? In: Proceedings of the 17th International Conference on Mining Software Repositories, pp. 492–502.

Pirocanac, G., 2020. Test flakiness–one of the main challenges of automated testing. https://testing.googleblogcom/2020/12/test-flakiness-one-of-main-challenges.html.
Pontillo, V., Palomba, F., Ferrucci, F., 2021. Toward static test flakiness prediction: a feasibility study. In: Proceedings of the 5th International Workshop on Machine Learning Techniques for Software Quality Evolution, pp. 19–24.
Sennrich, R., Haddow, B., Birch, A., 2015. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909.
Shi, A., Gyori, A., Legunsen, O., Marinov, D., 2016. Detecting assumptions on deterministic implementations of non-deterministic specifications. In: 2016 IEEE International Conference on Software Testing, Verification and Validation (ICST). IEEE, pp. 80–90.
Silva, D., Teixeira, L., d'Amorim, M., 2020. Shake it! detecting flaky tests caused by concurrency with shaker. In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, pp. 301–311.
Tahir, A., Rasheed, S., Dietrich, J., Hashemi, N., Zhang, L., 2023. Test flakiness' causes, detection, impact and responses: a multivocal review. J. Syst. Software 206, 111837.
Tang, L., Rajan, S., Narayanan, V.K., 2009. Large scale multi-label classification via metalabeler. In: Proceedings of the 18th International Conference on World Wide Web, pp. 211–220.
Tao, Y., Kim, J., Kim, S., Xu, C., 2014. Automatically generated patches as debugging aids: a human study. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 64–74.
Teng, Z., Zhang, Y., 2017. Head-lexicalized bidirectional tree LSTMS. Trans. Assoc. Comput. Ling. 5, 163–177.
Velez, M., Jamshidi, P., Sattler, F., Siegmund, N., Apel, S., Kästner, C., 2020. ConfigCrusher: towards white-box performance analysis for configurable systems. Autom. Software Eng. 27, 265–300.
Velez, M., Jamshidi, P., Siegmund, N., Apel, S., Kästner, C., 2021. White-box analysis over machine learning: modeling performance of configurable systems. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). IEEE, pp. 1072–1084.
Wang, R., Ye, S., Li, K., Kwong, S., 2021. Bayesian network based label correlation analysis for multi-label classifier chain. Inf. Sci. 554, 256–275.
Wehrmann, J., Cerri, R., Barros, R., 2018. Hierarchical multi-label classification networks. In: International Conference on Machine Learning. PMLR, pp. 5075–5084.
Winter, E., Bowes, D., Counsell, S., Hall, T., Haraldsson, S., Nowack, V., Woodward, J., 2023. How do developers really feel about bug fixing? Directions for automatic program repair. IEEE Trans. Software Eng. 49 (4), 1823–1841. https://doi.org/10.1109/TSE.2022.3194188
Wu, Q., Tan, M., Song, H., Chen, J., Ng, M.K., 2016. Ml-forest: a multi-label tree ensemble method for multi-label classification. IEEE Trans. Knowl. Data Eng. 28 (10), 2665–2680.
Wu, T., Huang, Q., Liu, Z., Wang, Y., Lin, D., 2020. Distribution-balanced loss for multi-label classification in long-tailed datasets. In: Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part IV 16. Springer, pp. 162–178.
Zhang, L., Radnejad, M., Miranskyy, A., 2023. Identifying flakiness in quantum programs. In: 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). IEEE, pp. 1–7.
Zhao, M., Hamarneh, G., 2019. Tree-LSTM: using LSTM to encode memory in anatomical tree prediction from 3D images. In: International Workshop on Machine Learning in Medical Imaging. Springer, pp. 637–645.
Zhou, X., Han, D., Lo, D., 2021. Assessing generalizability of codeBERT. In: 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 425–436. https://doi.org/10.1109/ICSME52107.2021.00044